Universidade de São Paulo
Escola de Artes, Ciências e Humanidades
Sistemas de Informação

Relatório do Exercício-Programa: Shoot 'em up com aplicações de Computação Orientada a Objeto

ACH2003 - Computação Orientada a Objetos

Gustavo Neves Moraes — NUSP: 16986195
Vinícius Cunha Pereira — NUSP: 16973242

Prof. Dr. Daniel Cordeiro

São Paulo
2026

Sumário

1 Vídeo de Demonstração **2**

2 Crítica ao Código Original **3**
2.1 Ausência de modelagem orientada a objetos 3
2.2 Representação das entidades por vetores paralelos 4
2.3 Consequências da arquitetura adotada 5
2.4 Impacto na manutenção e evolução do sistema 6

3 Arquitetura da reorganização do projeto **8**
3.1 Visão geral da hierarquia de classes adotadas 8
3.2 Interfaces em `game.core` 10
3.3 Por que `GameObject` e `Entity` são classes separadas 11
3.4 Um único algoritmo de colisão para todos os pares 12
3.5 Ciclo de vida do jogo 13
3.6 Uso de Coleções Java 14

4 Novas Funcionalidades e robustez da implementação **16**
4.1 Power-ups 16
4.2 Chefes de fase 17
4.3 Sistema de fases (`GameConfigParser` e `PhaseConfigParser`) 18
4.4 Robustez do código (exceções e logging) 19
4.4.1 Exceções 20
4.4.2 Logging 21

1 Vídeo de Demonstração

Vídeo disponível em: `https://youtu.be/ETJPdSAlrB4`

2 Crítica ao Código Original

O `Main.java` fornecido implementa corretamente o funcionamento do jogo, porém sua estrutur segue um modelo inteiramente procedural, apesar de ter sido desenvolvido em Java. Da maneira que foi aplicado o código, conceitos fundamentais da programação orientada a objetos, como encapsulamento, abstração, herança, polimorfismo e o uso de coleções, praticamente não são explorados. Em vez disso, toda a lógica da aplicação é concentrada em um único método, utilizando variáveis primitivas e vetores para representar as entidades do jogo. Em suma, pode ser comparado a um código de alguém que sabe o básico de Linguagem C e decidiu se aventurar no Java.

Embora essa abordagem seja suficiente para um projeto pequeno, ela dificulta a organização do código, reduz seu potencial de reutilização e torna sua evolução muito mais trabalhosa.

2.1 Ausência de modelagem orientada a objetos

Toda a lógica do jogo está concentrada na classe `Main`, principalmente dentro do método `main()`, auxiliado apenas por alguns métodos utilitários estáticos. Não existe uma modelagem que represente os conceitos do domínio do jogo, como jogador, inimigos, projéteis ou explosões, mesmo sendo entidades que possuem características e comportamentos próprios.

Como consequência, o estado de cada entidade é representado por variáveis primitivas independentes, manipuladas diretamente ao longo do programa, como no caso do jogador:

```
1 int player_state = ACTIVE;
2 double player_X = GameLib.WIDTH / 2;
3 double player_Y = GameLib.HEIGHT * 0.90;
4 double player_VX = 0.25;
5 double player_VY = 0.25;
6 double player_radius = 12.0;
7 double player_explosion_start = 0;
8 double player_explosion_end = 0;
9 long player_nextShot = currentTime;
```

Código 1: Variáveis primitivas sem uma classe que represente a entidade (Main.java)

Essa representação impede que os dados e o comportamento relacionados ao jogador sejam reunidos em uma única estrutura. Como não há encapsulamento, qualquer trecho do `main()` pode acessar ou modificar diretamente variáveis como `player_X`, `player_Y` ou `player_state`, sem qualquer mecanismo de controle ou validação.

Além disso, concentrar toda a lógica da aplicação em um único método faz com que responsabilidades bastante diferentes, como processamento de entrada, movimentação, geração de entidades, detecção de colisões e desenho, permaneçam fortemente interligados. Essa organização dificulta a manutenção do código e aumenta o impacto de qualquer simples alteração realizada no sistema.

2.2 Representação das entidades por vetores paralelos

A ausência de objetos torna-se ainda mais evidente nas entidades que possuem múltiplas instâncias, como inimigos e projéteis. Em vez de cada inimigo ser representado por um objeto contendo seus próprios atributos, o programa utiliza diversos vetores paralelos, nos quais cada posição corresponde implicitamente à mesma entidade.

O inimigo do tipo 1, por exemplo, é representado pelos seguintes vetores:

```
int    [] enemy1_states          = new int[10];
double [] enemy1_X               = new double[10];
double [] enemy1_Y               = new double[10];
double [] enemy1_V               = new double[10];
double [] enemy1_angle           = new double[10];
double [] enemy1_RV              = new double[10];
double [] enemy1_explosion_start = new double[10];
double [] enemy1_explosion_end   = new double[10];
long   [] enemy1_nextShoot       = new long[10];
```

Código 2: Representação do Inimigo tipo 1 (Main.java)

O mesmo padrão é repetido para os demais tipos de inimigo. Dessa forma, atributos que pertencem conceitualmente ao mesmo objeto ficam espalhados em diferentes estruturas, dependendo do índice do vetor para manter sua associação.

Essa escolha também impede o reaproveitamento de código entre entidades que compartilham características semelhantes. Conceitos comuns, como posição, velocidade, estado e remoção ao sair da tela, acabam sendo implementados repetidamente para cada conunto de vetores, em vez de serem centralizados em uma classe base.

Outro efeito dessa representação é a necessidade de gerenciar manualmente as posições disponíveis dentro dos vetores. Sempre que uma nova entidade precisa ser criada, o programa realiza uma busca linear para localizar um índice livre, como mostrado a seguir:

```
public static int findFreeIndex(int [] stateArray){
    int i;
    for(i = 0; i < stateArray.length; i++){
        if(stateArray[i] == INACTIVE) break;
    }
    return i;
}
```

Código 3: Busca manual por um índice livre no vetor (Main.java)

Além disso, o uso de vetores de tamanho fixo impõe uma limitação artificial ao número de entidades existentes simultaneamente no jogo. Quando todos os índices estão ocupados, novas entidades simplesmente deixam de ser criadas.

Na prática, esse método (`findFreeIndex`) busca reproduzir manualmente uma funcionalidade que já é oferecida pelas coleções da biblioteca padrão do Java, este tópico em específico sobre o uso das coleções de Java será melhor elaborado na Sessão 3.

2.3 Consequências da arquitetura adotada

A representação das entidades por meio de variáveis primitivas e vetores paralelos afeta diretamente toda a implementação do jogo. Como não existe uma abstração comum entre os diferentes tipos de entidades, operações semelhantes precisam ser reimplementadas para cada caso específico, resultando em grande quantidade de código duplicado (o que nunca é positivo).

Esse problema aparece na detecção de colisões. O programa possui vários laços distintos para verificar colisões entre diferentes pares de entidades (como jogador e projéteis inimigos, jogador e inimigos, entre outros). Embora trabalhem sobre vetores diferentes, todos executam exatamente a mesma sequência de operações (calcular a distância entre os objetos e compará-la com a soma de seus raios).

```
for(int i = 0; i < e_projectile_states.length; i++){
    double dx = e_projectile_X[i] - player_X;
    double dy = e_projectile_Y[i] - player_Y;
    double dist = Math.sqrt(dx * dx + dy * dy);
    if(dist < (player_radius + e_projectile_radius) * 0.8){
        player_state = EXPLODING;
        player_explosion_start = currentTime;
        player_explosion_end = currentTime + 2000;
    }
}

for(int i = 0; i < enemy1_states.length; i++){
    double dx = enemy1_X[i] - player_X;
    double dy = enemy1_Y[i] - player_Y;
    double dist = Math.sqrt(dx * dx + dy * dy);
    if(dist < (player_radius + enemy1_radius) * 0.8){
        player_state = EXPLODING;
        player_explosion_start = currentTime;
        player_explosion_end = currentTime + 2000;
    }
}
```

Código 4: Exemplos de laços de colisão presentes no código original (Main.java)

A única diferença entre esses trechos está na origem dos dados utilizados. A lógica da colisão permanece exatamente a mesma, sendo apenas adaptada para cada conjunto de vetores adequedo. Uma representação baseada em objetos permitiria aplicar o mesmo algoritmo de colisão a qualquer entidade que fornecesse informações como posição e raio, eliminando essa repetição.

O mesmo padrão também aparece na atualização das entidades e no desenho da cena. A movimentação dos projéteis do jogador e dos projéteis inimigos, por exemplo, é praticamente idêntica, alterando apenas a direção em que deixam a tela e os vetores utilizados.

```
for(int i = 0; i < projectile_states.length; i++){
    if(projectile_states[i] == ACTIVE){
        if(projectile_Y[i] < 0) {
```

```
            projectile_states[i] = INACTIVE;
        } else {
            projectile_X[i] += projectile_VX[i] * delta;
            projectile_Y[i] += projectile_VY[i] * delta;
        }
    }
}

for(int i = 0; i < e_projectile_states.length; i++){
    if(e_projectile_states[i] == ACTIVE){
        if(e_projectile_Y[i] > GameLib.HEIGHT) {
            e_projectile_states[i] = INACTIVE;
        } else {
            e_projectile_X[i] += e_projectile_VX[i] * delta;
            e_projectile_Y[i] += e_projectile_VY[i] * delta;
        }
    }
}
```

Código 5: Atualização dos projéteis do jogador e dos projéteis inimigos (Main.java)

Situações semelhantes ocorrem em diversos outros pontos do programa, como no desenho das entidades e na atualização dos diferentes tipos de inimigo. Sempre que um novo tipo de objeto é introduzido, torna-se necessário escrever novos laços muito parecidos com os já existentes, alterando apenas nomes de vetores e algumas constantes.

2.4 Impacto na manutenção e evolução do sistema

Além dos problemas de organização, a estrutura do código limita a evolução do jogo. Como toda a lógica está concentrada em um único método e cada tipo de entidade possui sua própria implementação, qualquer funcionalidade nova exige alterações em diversos pontos diferentes do programa.

Essa dificuldade torna-se ainda mais evidente no gerenciamento das entidades. Todos os vetores possuem tamanho fixo, definido manualmente durante sua criação. Quando não há mais posições disponíveis, novas entidades simplesmente deixam de ser criadas.

```
int free = findFreeIndex(projectile_states);
if(free < projectile_states.length){
    projectile_X[free] = player_X;
    projectile_Y[free] = player_Y - 2 * player_radius;
    projectile_VX[free] = 0.0;
    projectile_VY[free] = -1.0;
    projectile_states[free] = ACTIVE;
    player_nextShot = currentTime + 100;
}
// se free == projectile_states.length, o tiro simplesmente não acontece
```

Código 6: Projetil descartado quando todos os índices do vetor estão ocupados (Main.java)

Essa limitação obriga o programa a manter capacidades máximas definidas arbitrariamente e a controlar manualmente quais posições dos vetores estão livres ou ocupadas. Em aplicações maiores, esse tipo de gerenciamento aumenta a complexidade do código e dificulta a expansão do sistema.

Por fim, praticamente todas as responsabilidades do jogo permanecem concentradas dentro do método `main()`. A ausência de uma divisão clara entre essas responsabilidades reduz a modularidade da aplicação e faz com que alterações em uma funcionalidade tenham maior probabilidade de impactar outras partes do sistema.

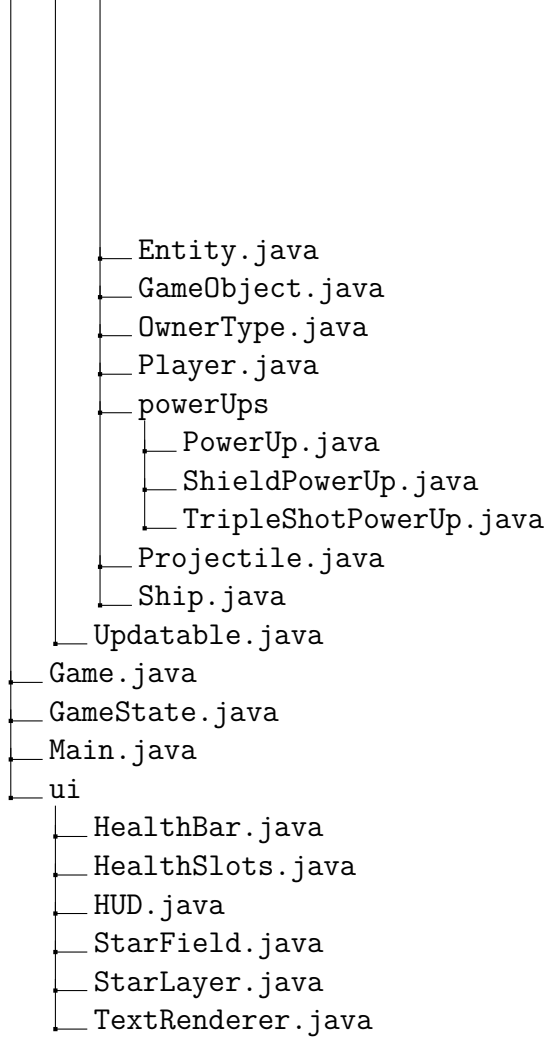
3 Arquitetura da reorganização do projeto

Esta seção apresenta uma nova organização para o mesmo projeto mas de modo a evitar os erros que foram apontados na Sessão 2. Em vez de descrever apenas o resultado final, a ideia é mostrar também o raciocínio por trás de cada decisão, justificando a existência de classes, herança, interfaces e como cada escolha resolve algum dos problemas discutidos anteriormente.

3.1 Visão geral da hierarquia de classes adotadas

Para substituir a implementação procedural do jogo, o projeto foi reorganizado em uma estrutura baseada em classes e interfaces, separando responsabilidades e agrupando comportamentos semelhantes. A organização final dos pacotes ficou da seguinte forma:

```
game
├── config
│   ├── GameConfig.java
│   ├── GameConfigParser.java
│   ├── phase
│   │   ├── PhaseConfigParser.java
│   │   └── Phase.java
│   └── spawn
│       ├── BossKind.java
│       ├── BossSpawnEntry.java
│       ├── EnemyKind.java
│       ├── EnemySpawnEntry.java
│       ├── EntitySpec.java
│       ├── PowerUpKind.java
│       ├── PowerUpSpawnEntry.java
│       └── SpawnManager.java
└── core
    ├── collision
    │   ├── Collidable.java
    │   └── CollisionManager.java
    ├── Damageable.java
    ├── Drawable.java
    └── entities
        └── enemies
            ├── Boss.java
            ├── BossType1.java
            ├── BossType2.java
            ├── Enemy.java
            ├── EnemyType1.java
            └── EnemyType2.java
```



- `game.core` - reúne interfaces utilizadas por diferentes partes do jogo (`Drawable`, `Updatable`, `Damageable`).
- `game.core.collision` - concentra a lógica relacionada ao sistema de colisão (`Collidable` e `CollisionManager`).
- `game.core.entities` - contém as entidades que representam os elementos do jogo, com os subpacotes `enemies` (inimigos comuns e chefes) e `powerUps` (os dois tipos de power-up).
- `game.config` - reúne a leitura dos arquivos de configuração, com os subpacotes `spawn` (representação unificada das linhas de fase) e `phase` (controle de quando cada entidade deve surgir).
- `game.ui` - reúne os elementos de interface visual do jogo, como o `HUD` e o fundo de estrelas.

A hierarquia de classes das entidades (`game.core.entities`), por exemplo, pode ser representada por:

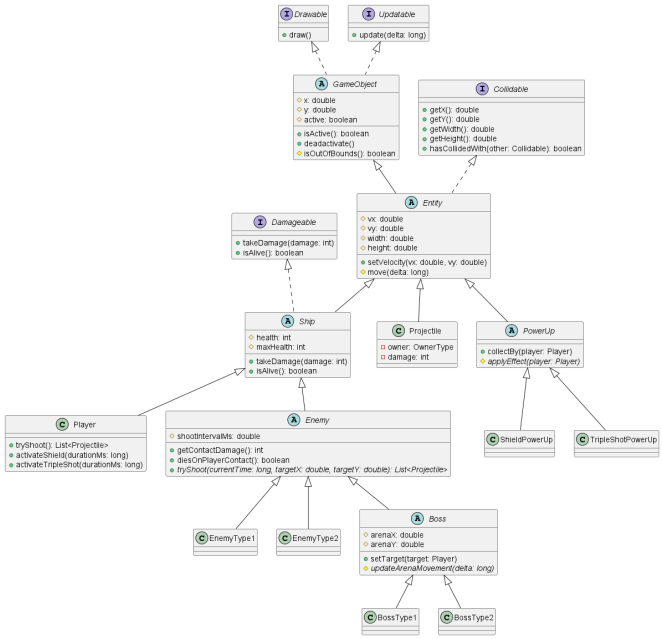


Figura 1: Hierarquia de classes de `core.entities`.

A hierarquia foi construída de forma que cada nível acrescente apenas as características específicas daquele conjunto de entidades (princípio classico de OO). Assim, informações comuns, como posição, movimentação ou vida, são definidas apenas um vez e reaproveitadas pelas subclasses, reduzindo duplicação de código (consequentemente reduzindo chances de bugs) e facilitando a manutenção da implementação.

3.2 Interfaces em `game.core`

As interfaces foram utilizadas para representar capacidades independentes das entidades do jogo. Em vez de concentrar todos os comportamentos em uma única classe base, cada interface define apenas uma responsabilidade específica (similar a um contrato), permitindo que cada classe implemente somente os comportamentos que são realmente necessários.

- `Drawable`. define a operação responsável pelo desenho da entidade na tela.
- `Updatable`. representa objetos cujo estado precisa ser atualizado a cada ciclo do jogo.
- `Damageable`. reúne as operações relacionadas ao sistema de dano e vida, sendo implementada apenas pelas entidades que podem sofrer dano.
- `Collidable`. define as informações necessárias para o sistema de colisão e disponibiliza um método padrão para verificar a sobreposição entre dois objetos.

Essa separação torna a arquitetura mais flexível, pois evita que uma classe precise implementar comportamentos que não fazem sentido para ela. Por exemplo, os elementos do `game.ui`, como o fundo de estrelas, implementam apenas `Drawable` e `Updatable`, sem participar do sistema de colisão nem possuir pontos de vida, já que essas capacidades não fazem sentido para um elemento puramente visual.

```
public interface Collidable {
    double getX();
    double getY();
    double getWidth();
    double getHeight();

    default boolean hasCollidedWith(Collidable other){
        double halfW1 = getWidth() / 2.0;
        double halfH1 = getHeight() / 2.0;
        double halfW2 = other.getWidth() / 2.0;
        double halfH2 = other.getHeight() / 2.0;

        return Math.abs(getX() - other.getX()) < (halfW1 + halfW2)
            && Math.abs(getY() - other.getY()) < (halfH1 + halfH2);
    }
}
```

Código 7: contrato e comprtamento padrão da interface `Collidable` (Collidable.java)

Por ser um método `default`, qualquer classe que implemente `Collidable` já ganha automaticamente a lógica de detecção de sobreposição entre retângulos, sem precisar reimplementar.

3.3 Por que `GameObject` e `Entity` são classes separadas

A classe `GameObject` reúne apenas as informações presentes em qualquer objeto exibido na tela, como posição, estado de atividade e a verificação de quando um objeto saiu dos limites da tela:

```
public abstract class GameObject implements Drawable, Updatable {
    protected double x;
    protected double y;
    protected boolean active;
    ...
    protected boolean isOutOfBounds(){
        return x < - OUT_OF_BOUNDS_MARGIN
            || x > GameLib.WIDTH + OUT_OF_BOUNDS_MARGIN
            || y < - OUT_OF_BOUNDS_MARGIN
            || y > GameLib.HEIGHT + OUT_OF_BOUNDS_MARGIN;
    }
}
```

Código 8: Responsabilidades mínimas comuns a qualquer objeto do jogo (GameObject.java)

A classe `Entity` estende essa estrutura (ou seja `Entity` é um `GameObject`) adicionando atributos relacionados à movimentação e às colisões, como velocidade e dimensões, além de implementar `Collidable`. Parte da estrutura de Entity a seguir:

```
public abstract class Entity extends GameObject implements Collidable {
    protected double vx, vy;
    protected double width, height;
    ...
    @Override
    public void update(long delta){
        move(delta);

        if (isOutOfBounds()){
            deadactivate();
        }
    }
}
```

Código 9: `Entity` reparoveita isOutOfBounds() da sua superclasse (Entity.java)

Repare que `update` cuida para que qualquer entidade que saia da área de jogo (um projétil ou um inimigo comum, por exemplo) seja desativada automaticamente por esse comportamento herdado (`isOutOfBounds`), sem que cada subclasse precise implementar essa verificação por conta própria.

Essa divisão permite que diferentes tipos de entidade compartilhem toda a lógica básica de movimentação sem carregar responsabilidades que não lhes pertencem. Por exemplo, projéteis e power-ups herdam o comportamento de movimento definido em `Entity`, mas não possuem atributos relacionados à vida ou dano, que só passam a existir na classe `Ship`, utilizada pelo jogador e pelos inimigos (para melhor visualizaçãodo exemplo basta ver a Figura 1).

3.4 Um único algoritmo de colisão para todos os pares

A interface `Collidable` permite unificar a lógica de colisão em um único algoritmo capaz de operar sobre qualquer objeto que implemente essa interface. Dessa forma, o mesmo mecanismo pode ser utilizado para verificar colisões entre jogadores, inimigos, projéteis e power-ups. A classe `CollisionManager` concentra todas essas verificações:

```
public void checkCollisions(Player player, List<Enemy> enemies, List<
    Projectile> projectiles){
    checkProjectilesAgainstEnemies(projectiles, enemies);
    checkProjectilesAgainstPlayer(projectiles, player);
    checkEnemiesAgainstPlayer(enemies, player);
}

public void checkPowerUpPickup(Player player, List<PowerUp> powerUps){
    for (PowerUp p : powerUps){
        if (p.isActive() && p.hasCollidedWith(player)) {
            p.collectBy(player);
```

```
11          }
12      }
13  }
```

Código 10: Alguns métodos de `CollisionManager` operando sobre listas de classes `Collidable` (CollisionManager.java)

Essa abordagem elimina a necessidade de criar uma implementação específica para cada combinação de entidades, como ocorria no código original, no qual vários laços praticamente idênticos verificavam colisões entre pares diferentes de vetores. Sempre que um novo objeto participa do sistema de colisão, basta que ele implemente `Collidable` para que passe a utilizar o algoritmo já existente, sem exigir nenhuma alteração no `CollisionManager`.

3.5 Ciclo de vida do jogo

Além da organização interna de cada entidade, o comportamento do jogo como um todo também é modelado explicitamente através de um conjunto finito de estados, representados pelo enum `GameState`. Cada estado define o que está acontecendo na tela naquele momento (carregando, jogando, em transição de fase, entre outros), e as transições entre eles são disparadas por eventos específicos, como a derrota do jogador, a conclusão de uma fase, ou o pressionamento de uma tecla.

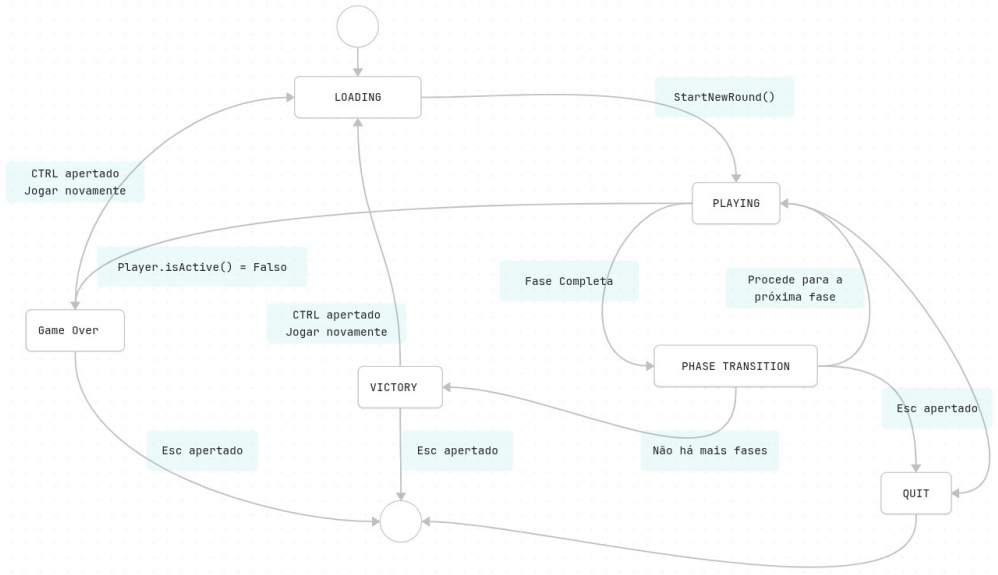


Figura 2: Diagrama de estados do jogo

Em suma, funciona da seguinte maneira, o jogo inicia em `LOADING`, e assim que uma nova rodada é montada (`startNewRound`), passa para `PLAYING` (estado em que a maior parte da lógica de atualização e desenho acontece). Ao derrota o chefe de uma

fase, o jogo transita brevemente para `PHASE_TRANSITION`, exibindo uma mensagem de conclusão antes de avançar para a próxima fase (retornando a `PLAYING`) ou, caso não haja mais fases, para `VICTORY`. Já a derrota do jogador (quando `Player.isActive()` se torna falso) leva diretamente a `GAME_OVER`, e o pressionamento da tecla `ESC` em qualquer momento do jogo leva a `QUIT`, encerrando a execução.

Essa modelagem evita que o estado do jogo seja controlado por múltiplas variáveis booleanas soltas, que poderiam entrar em combinações inconsistentes entre si. Com um único enum representando o estado atual, cada parte do código pode consultar exatamente em qual estado o jogo se encontra e reagir de forma correspondente.

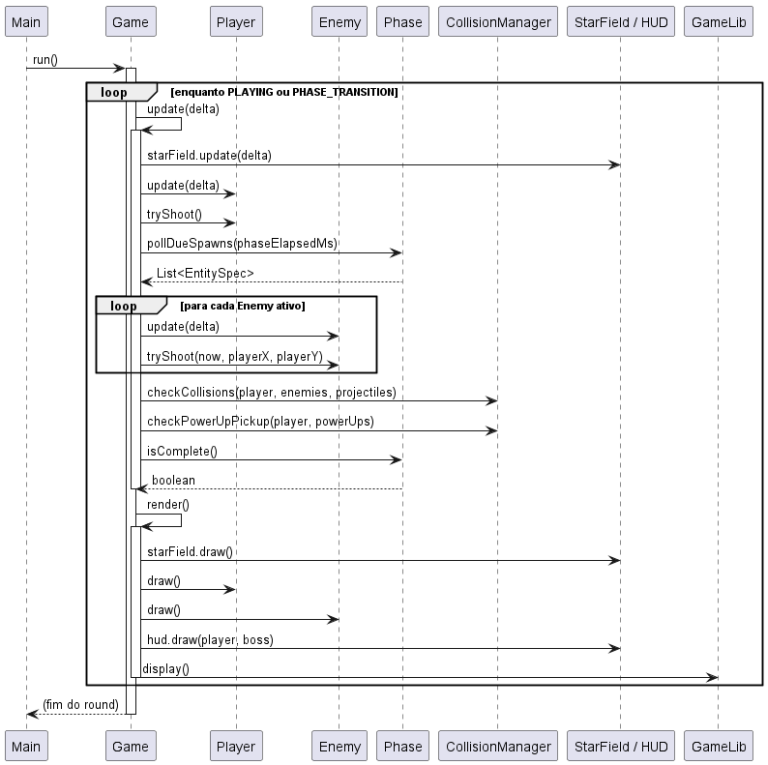


Figura 3: Loop principal

3.6 Uso de Coleções Java

Enquanto o `Main.java` original mantinha cada conjunto de entidades em vetores de tamanho fixo, controlando manualmente quais posições estavam ocupadas ou livres, a nova abordagem aplicada na nova organização substitui essa implementação pelo uso das coleções do Java. A classe `Game` passou a armazenar inimigos, projéteis e power-ups em listas.

```
1 private final List<Enemy> enemies;
2 private final List<Projectile> projectiles;
3 private final List<PowerUp> powerUps;
```

```
...
this.enemies = new LinkedList<>();
this.projectiles = new LinkedList<>();
this.powerUps = new LinkedList<>();
```

Código 11: Gerenciamento de entidades

Como as coleções do Java são estruturas dinamicas, não existe mais um limite pré-estabelecido para o número de entidades simultâneas em cena, nem a necessidade de uma busca manual por um índice livre (como era feito no código original). Basta adicionar um novo elemento à coleção correspondente quando uma entidade é criada.

A remoção de entidades inativas também deixou de depender de verificações manuais de estado espalhadas pelo código, sendo concentrada em uma única chamada por coleção a cada ciclo de atualização.

```
enemies.removeIf(e -> !e.isActive());
projectiles.removeIf(p -> !p.isActive());
powerUps.removeIf(p -> !p.isActive());
```

Código 12: Remoção de entidades inativas via `removeIf` (Game.java)

Dessa forma, garante-se que nenhuma entidade inativa permaneça ocupando espaço indefinidamente, assim evitando preocupações quanto ao vazamento de memória.

Vale ressaltar que o uso de coleções não se restringiu ao gerenciamento das entidades em jogo. A classe `Phase`, responsável por controlar a aparição de inimigos, chefes e power-ups ao longo de uma fase, também é construída sobre uma lista.

```
private final List<EntitySpec> schedule; // ordered by whenMs

public List<EntitySpec> pollDueSpawns(long elapsedMs){
    List<EntitySpec> due = new ArrayList<>();

    while (nextIndex < schedule.size() && schedule.get(nextIndex).
        getWhenMs() <= elapsedMs) {
        due.add(schedule.get(nextIndex));
        nextIndex++;
    }

    return due;
}
```

Código 13: Utilização de lista para agendar as aparições da fase (Phase.java)

Nesse caso, a lista ordenada por instante de aparição permite que o método `pollDueSpawns` percorra o cronograma da fase de forma incremental, devolvendo apenas as entidades cujo instante de aparição já foi atingido.

4 Novas Funcionalidades e robustez da implementação

4.1 Power-ups

Foram implementados dois tipos de power-up, ambos representados pela classe abstrata `PowerUp`, que estende `Entity` e reutiliza toda a lógica de movimento já existente na hierarquia, definida por uma velocidade vertical fixa no próprio construtor.

```
1  public abstract class PowerUp extends Entity {
2      protected PowerUp(double x, double y, double width, double height,
           double fallSpeed) {
3          super(x, y, width, height);
4          setVelocity(0, fallSpeed);
5      }
6
7      public final void collectBy(Player player) {
8          if (!isActive()) return;
9
10         applyEffect(player);
11         deadactivate();
12     }
13
14     protected abstract void applyEffect(Player player);
15 }
```

Código 14: `PowerUp` concentra a coleta em um único método `final` (PowerUp.java)

As seguintes regras são comuns a todo power-up e não podem ser alteradas por engano em qualquer sub-classe (por isso o `final`):

- Um power-up só pode ser coletado uma única vez.
- Ele se desativa imediatamente após ser coletado.

No entanto, o efeito em si (`applyEffect`) varia conforme o tipo de power-up. Esses foram os Power-ups implementados:

- **`ShieldPowerUp`**: Concede ao jogador um escudo temporário de 5 segundos, tornando-o invulnerável a qualquer dano. A habilidade tem o efeito visual adicional de deixar o player azul, substituindo temporariamente sua cor original.
- **`TripleShotPowerUp`**: Faz o jogador disparar três projéteis em leque (dispersão de 15 graus) a cada tiro, por 8 segundos.

Vale notar que a escolha de cada efeito exigiu também a criação de métodos correspondentes na própria classe `Player`, responsáveis por ativar, consultar e expirar cada efeito ao longo do tempo. Exemplo:

```java
public void activateShield(long durationMs) {
    shieldTimeRemaining = durationMs;
}

public boolean isShielded() {
    return shieldTimeRemaining > 0;
}

public void activateTripleShot(long durationMs) {
    tripleShotTimeRemaining = durationMs;
}

public boolean hasTripleShot() {
    return tripleShotTimeRemaining > 0;
}
```

Código 15: Métodos adicionados a `Player` para suportar os efeitos dos power-ups (Player.java)

Cada efeito guarda apenas um contador de tempo restante, decrementado a cada quadro em `Player.update`, e é consultado no momento certo por outro método já existente, por exemplo, `isShielded()` é verificado em `takeDamage` (anulando qualquer dano recebido enquanto o escudo estiver ativo) e `hasTripleShot()` é verificado em `tryShoot` (decidindo se dois projéteis extras em leque devem ser adicionados ao disparo).

4.2 Chefes de fase

Foram implementados dois chefes, ambos estendendo a classe abstrata `Boss`, que por sua vez estende `Enemy`. Uma diferença entre os inimigos comuns e os chefes está no fato em que todo chefe nasce fora da tela e desliza até uma posição específica da arena antes de assumir seu comportamento próprio.

- **`BossType1`** Patrulha lateralmente entre dois limites fixos dentro da arena, sem reagir à posição do jogador, e dispara periodicamente um leque de três projéteis retos para baixo.
- **`BossType2`**: Descreve um movimento senoidal de oscilando verticalmente enquanto deriva lateralmente pela arena, disparando um leque de projéteis exatamente uma vez a cada ciclo de mergulho.

A classe `Boss` concentra o comportamento comum de entrada na arena e delega a cada subclasse apenas o que muda depois disso:

```java
@Override
public void update(long delta) {
    if (updateExploding(delta)) return;

```

```
5       if (entering) {
6           updateEntrance(delta);
7       } else {
8           super.update(delta);
9           updateArenaMovement(delta);
10      }
11  }
12
13  // Each boss is responsible for defining its own movement pattern within
        the screen.
14  protected abstract void updateArenaMovement(long delta);
```

Código 16: `Boss` decide entre a entrada (comum) e o comportamento próprio de cada chefe (Boss.java)

Além disso, `Boss` sobrescreve dois comportamentos herdados de `Enemy`/`GameObject` para atender aos requisitos específicos de um chefe de fase: `isOutOfBounds` sempre retorna `false` (um chefe nunca é desativado por sair da tela), e `diesOnPlayerContact` também retorna `false` (diferentemente dos inimigos comuns, um chefe só pode ser derrotado atirando nele, nunca por colisão direta).

Ademais, A vida do chefe é exibida através de uma barra de vida (`HealthBar`) enquanto ele estiver ativo em cena, aparecendo e sumindo junto com ele.

4.3 Sistema de fases (`GameConfigParser` e `PhaseConfigParser`)

O jogo é configurado por dois níveis de arquivo texto, lidos em tempo de execução. Um arquivo principal, definindo a vida do jogador e a lista de arquivos de fase, e um arquivo por fase, definindo os inimigos, power-ups e o chefe daquela fase (como consta no próprio enunciado).

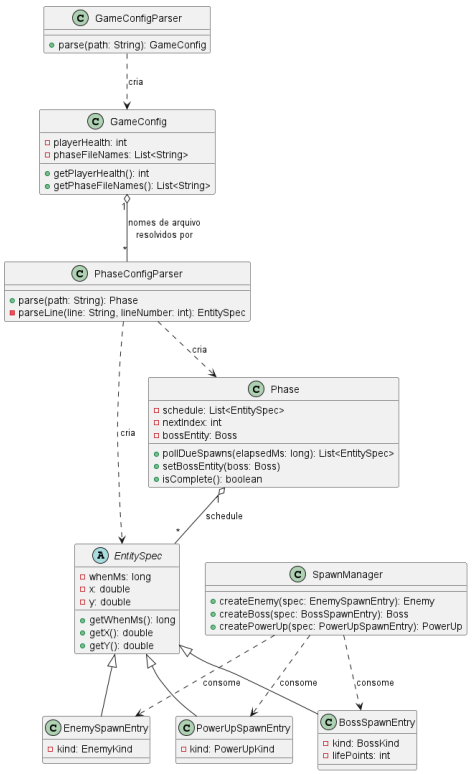


Figura 4: Diagrama do sistema de fases

A leitura desses arquivos é feita pelas classes `GameConfigParser` e `PhaseConfigParser`, que traduzem cada linha em um objeto do tipo `EntitySpec` (com as subclasses `EnemySpawnEntry`, `BossSpawnEntry` e `PowerUpSpawnEntry`), unificando os três tipos de entrada em uma única representação.

A tradução desses dados declarativos em entidades de fato jogáveis é responsabilidade da classe `SpawnManager`, com métodos `createBoss`, `CreatePowerUp` e `CreateEnemy`.

Dessa forma, o restante do jogo (a classe `Game`) precisa apenas consultar o `SpawnManager` com o `EntitySpec` lido do arquivo de configuração, e a instância concreta correta é devolvida.

4.4 Robustez do código (exceções e logging)

A leitura dos arquivos de configuração citado na sessão anterior é o ponto do projeto com maior exposição a erros em tempo de execução. Por isso, o módulo de configuração (`game.config` e seus subpacotes) foi tratado com atenção especial, combinando exceções descritivas com o uso de `java.util.logging` para registrar o que aconteceu.

4.4.1 Exceções

Sempre que uma linha de um arquivo de configuração está malformada, o parser correspondente lança uma exceção com uma mensagem específica, indicando exatamente qual campo é o problema. Exemplo:

```
private int parseIntline(String line, String description){
    if (line == null)
        throw new IllegalStateException("Arquivo de configuração
            incompleto: faltando " + description + ".");

    try {
        return Integer.parseInt(line.trim());
    } catch (NumberFormatException e) {
        throw new IllegalStateException("Valor inválido para " +
            description + ": \"" + line + "\"", e);
    }
}
```

Código 17: Exceções descritivas (GameConfigParser.java)

Essas exceções, no entanto, são tratadas de duas formas diferentes dependendo da sua gravidade, o que evita que um erro pontual derrube o jogo inteiro desnecessariamente. Uma entrada isolada malformada dentro de um arquivo de fase (um `INIMIGO` ou `POWERUP` com tipo desconhecido/fora do padrão, por exemplo) é simplesmente ignorada, registrando um aviso e permitindo que o restante da fase continue normalmente, como mostra:

```
private void spawnDueEntities() {
    List<EntitySpec> due = currentPhase.pollDueSpawns(phaseElapsedMs);

    for (EntitySpec spec : due) {
        try {
            spawnEntity(spec);
        } catch (IllegalArgumentException e) {
            // a single malformed/unknown entry in the phase file shouldn
                't take
            // the whole game down; skip it and keep playing.
            LOGGER.log(Level.WARNING, "Skipping invalid spawn entry: " +
                spec, e);
        }
    }
}
```

Código 18: Uma entrada inválida não derruba a fase inteira (Game.java)

Já uma falha estrutural mais grave, como o próprio arquivo de configuração principal não sendo encontrado, é trata como fatal em um único ponto central, no laço principal do jogo, evitando que a aplicação trave com uma janela aberta e sem nenhuma explicação visível ao usuário.

```
try {
```

```
2       ...
3   } catch (Exception e) {
4       // last resort: without this, a startup/runtime failure would crash
            the thread
5       // after initGraphics() already opened the window, leaving a frozen,
6       // never-drawn screen with no visible clue why
7       LOGGER.log(Level.SEVERE, "Fatal error, aborting game.", e);
8   }
```

Código 19: Falhas fatais são registradas e encerram o jogo (Game.java)

4.4.2 Logging

O uso do `Logger` (de `java.util.logging`) contribui para uma melhor organização das informações geradas durante a execução do programa. Em vez de exibir todas as mensagens da mesma forma, o sistema permite classificá-las de acordo com sua importância, facilitando a identificação de eventos relevantes e de possíveis problemas.

Além de auxiliar durante o desenvolvimento e os testes, o uso de *logging* é considerado uma boa prática em projetos de software, pois torna a manutenção mais simples e o acompanhamento do comportamento da aplicação mais eficiente. Dessa forma, informações importantes podem ser registradas sem comprometer a clareza da saída principal do jogo, mantendo a experiência do usuário mais organizada e facilitando futuras correções ou melhorias.